AIML PORTFOLIO • ARTIFACT 2 • WORKSHOP 2.4

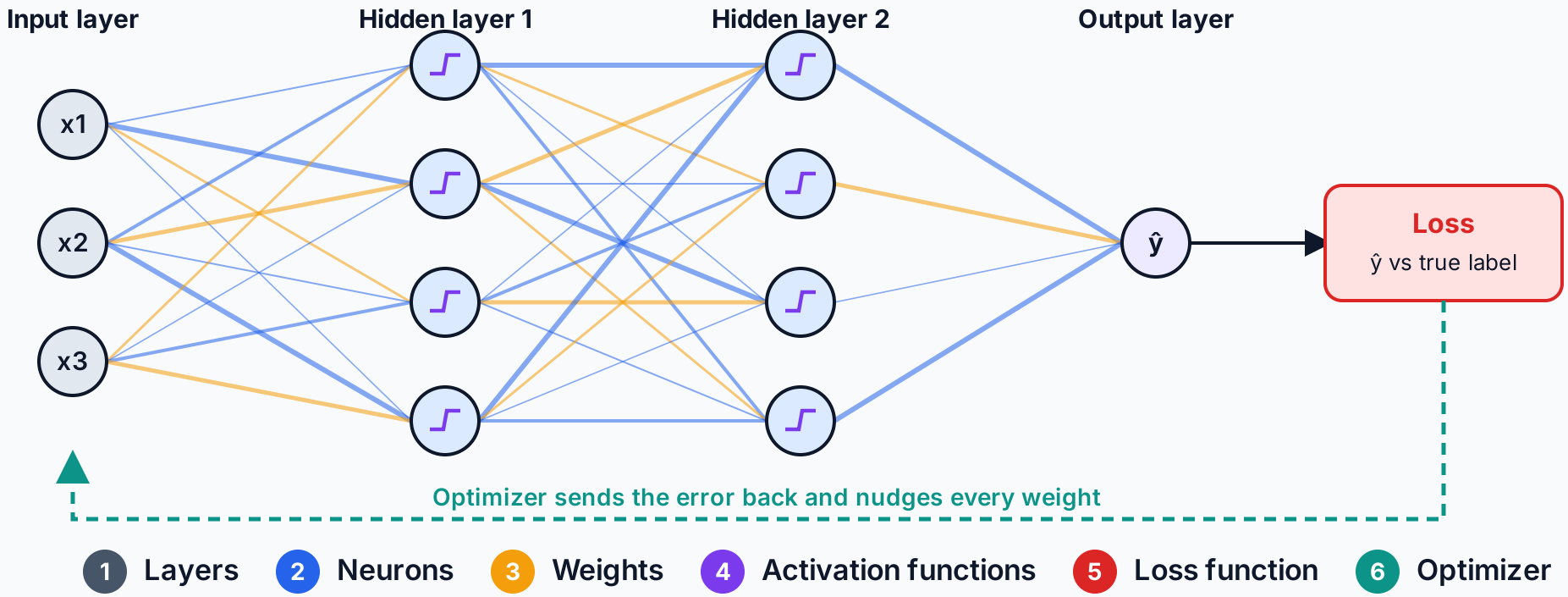
# Inside a Neural Network: Six Components, Explained Visually

Layers, neurons, weights, activation functions, loss functions and optimization algorithms, with results from my own experiments in the TensorFlow Playground.

Deepthi Pinninti | Indiana Wesleyan University | November 2026

THE BIG PICTURE

# How a neural network is put together




Data enters on the left as numbers, moves through the hidden layers, and leaves as a prediction. The loss scores that prediction, and the optimizer uses the score to improve the weights.

COMPONENT 1

# Layers

### Input layer

Receives the raw data, one node per feature. For an image that means one node per pixel. In the Playground it is the coordinates $x_1$ and $x_2$. It does no calculation.

### Hidden layers

Sit between input and output. Each layer takes the patterns found by the one before and combines them into more complex ones: edges, then shapes, then objects.

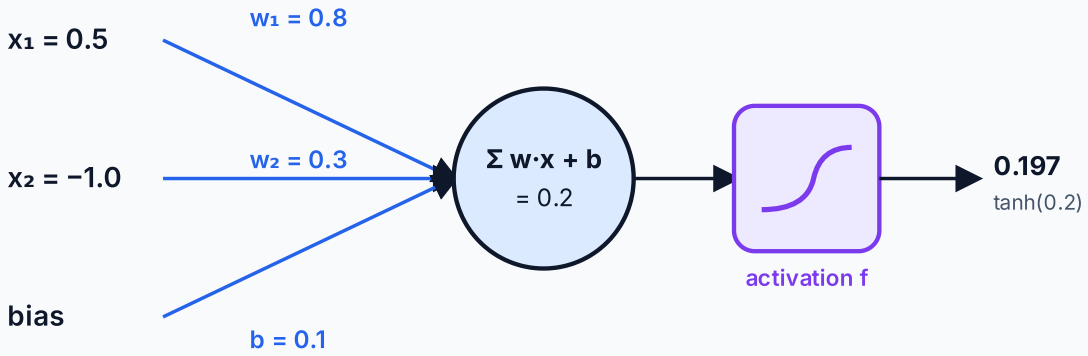
### Output layer

Produces the answer. One node with a sigmoid for yes/no, one node per class with softmax for several classes, or one plain number for regression.

**Why it matters:** "Deep" learning just means many hidden layers. In my Playground test on the circle data, a network with **no hidden layer** ended with a test loss of **0.503**, while one hidden layer of 4 neurons reached **0.010**.

COMPONENT 2

# Neurons




A neuron is a small calculator with three steps:

1. **Multiply** each input by its weight.
2. **Add** the results and a bias.
3. **Pass** the total through an activation function.

Example with the numbers shown: 0.5×0.8 + (−1.0)×0.3 + 0.1 = 0.2, and tanh(0.2) ≈ 0.197.

The output goes to every neuron in the next layer. A network is thousands of these tiny calculations working together.

COMPONENT 3

# Weights (and biases)

**Weights** are the numbers on the connections. A large weight means that input matters a lot to the neuron. A negative weight pushes the other way. The **bias** shifts the neuron's threshold.

Weights are what the network **learns**. They start random and are adjusted thousands of times during training. Everything the model knows is stored in them.

In the Playground, line **thickness** shows how large a weight is, and **colour** shows the sign (orange negative, blue positive).

### Counting parameters

My spiral network used 6 input features and two hidden layers of 8 neurons:

6×8 + 8×8 + 8×1 = **120 weights**

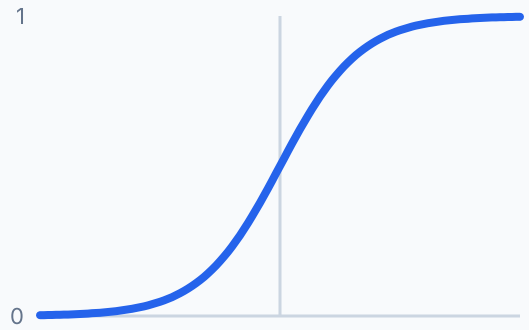
8 + 8 + 1 = **17 biases**

Total = **137 learned numbers**

Large language models have billions.

COMPONENT 4

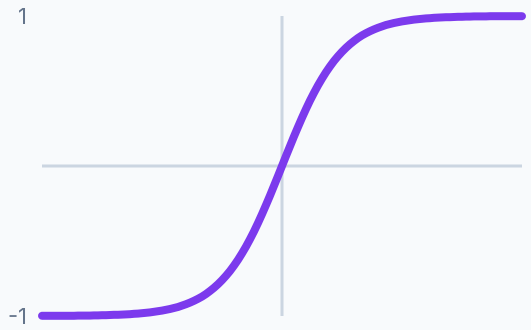
# Activation functions




**Sigmoid**

$1 / (1 + e^{-x})$

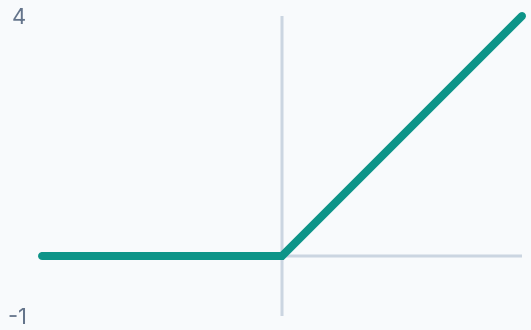
**Sigmoid** squashes any input into 0 to 1. Good for probabilities at the output, but it flattens at the ends, which slows learning.




**Tanh**

$(e^{x} - e^{-x}) / (e^{x} + e^{-x})$

**Tanh** squashes into −1 to 1 and is centred on zero. Works well in small networks, but also flattens at the ends.




**ReLU**

$\max(0, x)$

**ReLU** passes positives and blocks negatives. It is fast and does not flatten for positive values, so it is the common default in deep networks.

**Why needed:** without a bend (non-linearity), many layers collapse into one straight line. On the spiral data, with the same network and 503 epochs, test loss was **0.006 (ReLU)**, **0.065 (tanh)** and **0.293 (sigmoid)**.

COMPONENT 5

# Loss functions

The loss function is the **scoreboard**. It compares the network's prediction ŷ with the true answer y and returns one number: the bigger the number, the worse the prediction. Training means making it smaller.

### Two common choices

**Mean squared error** (regression)

$$L = (1/n)\ \Sigma\ (y - \hat{y})^2$$

**Cross-entropy** (classification)

$$L = -\ \Sigma\ y \cdot \log(\hat{y})$$

The Playground shows the loss for both the training data and the test data, plus a curve of how it falls.

### What I observed (circle data)

| Network | Train | Test |
|---|---|---|
| No hidden layer | 0.501 | 0.503 |
| 1 layer, 4 neurons | 0.007 | 0.010 |
| 2 layers, 4 + 2 | 0.001 | 0.004 |

Measured after about 500 epochs.

COMPONENT 6

# Optimization algorithms




The optimizer decides **how to change the weights** to reduce the loss. Gradient descent works like walking downhill in fog: feel which way the ground slopes, then take a step that way.

$$w \leftarrow w - \eta \cdot \partial L/\partial w$$

η (eta) is the **learning rate**, the step size. The gradient ∂L/∂w comes from backpropagation. The Playground uses mini-batch stochastic gradient descent; Adam and momentum are popular improvements.

| Learning rate | Test loss at ~500 epochs |
|---|---|
| 0.001 (too small) | 0.462 |
| 0.03 (good) | 0.004 |
| 3 (too large) | 0.718 |

PUTTING IT TOGETHER

# One training step, repeated thousands of times

1

### Forward pass

Data flows through layers, neurons and activations to produce a prediction.

→

2

### Measure loss

The loss function scores how far the prediction is from the true label.

→

3

### Backpropagate

The error is passed backward to work out each weight's share of the blame.

→

4

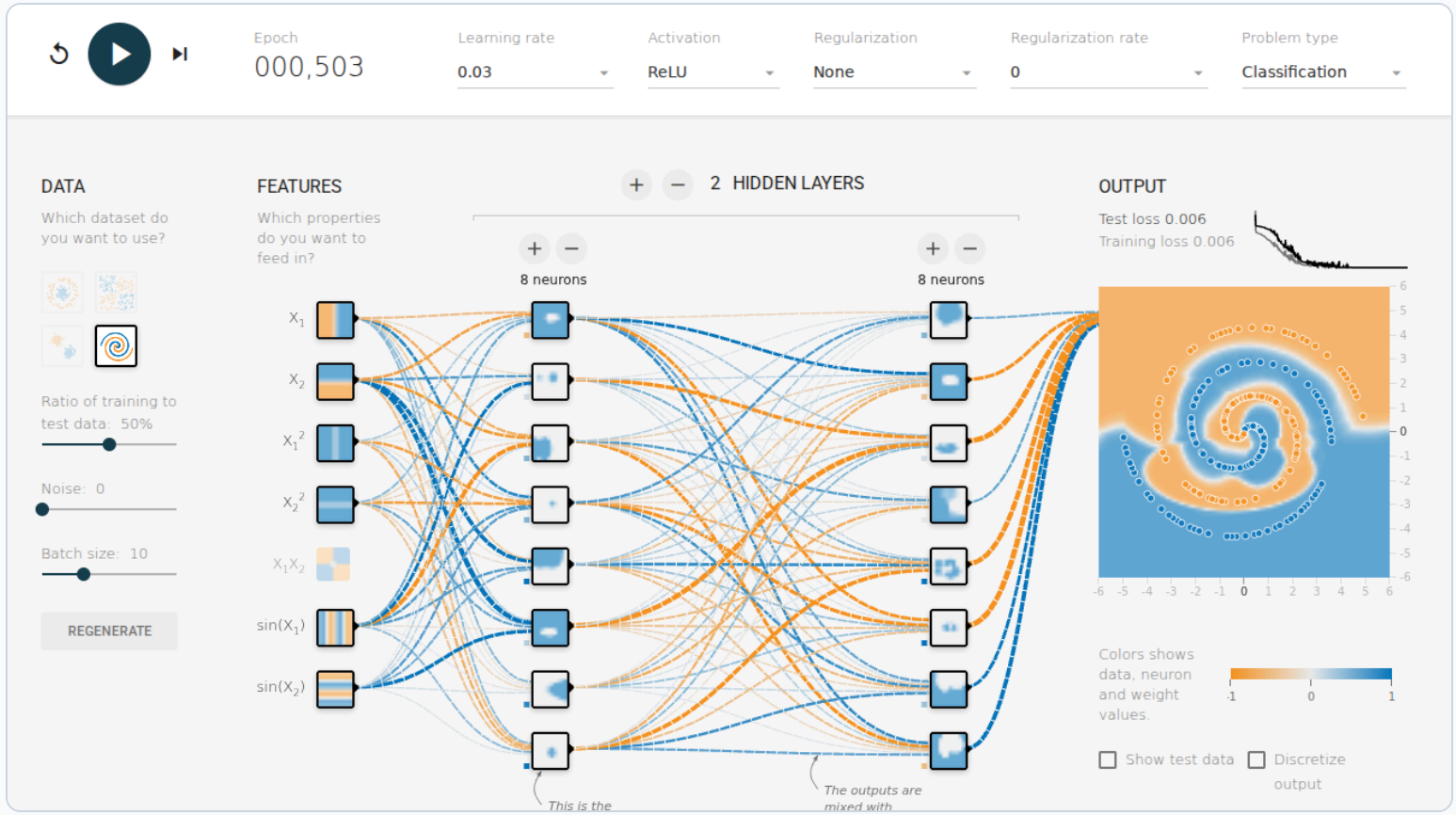
### Update weights

The optimizer moves each weight a small step to lower the loss.

One pass through all the training data is an **epoch**. The Playground's epoch counter shows this loop running, and in my experiments I let it run for about 500 epochs.

MY PRACTICE SETUP

# Training a model in the TensorFlow Playground




Spiral data, two hidden layers of 8 neurons, ReLU, after 503 epochs.

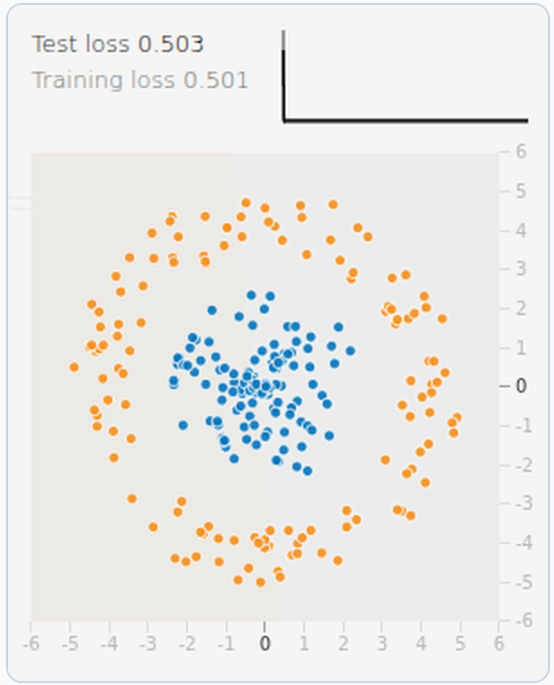
## What I varied

- Data: circle, XOR, spiral
- Noise: 0 to 40
- Hidden layers and neurons: 0 to 8 per layer
- Input features: x, y, $x^2$, $y^2$, sin
- Activation: tanh, ReLU, sigmoid
- Learning rate: 0.001 to 3

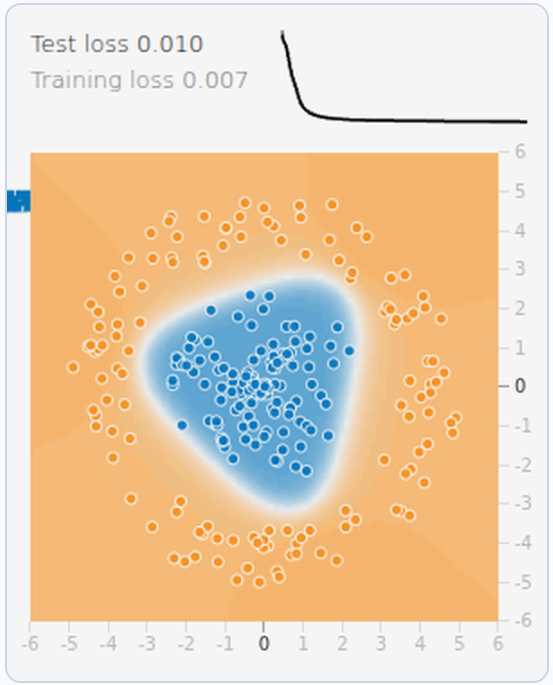
Each run used the same random seed and about 500 epochs, so differences come from the setting I changed. I recorded training and test loss.

PLAYGROUND EXPERIMENT 1

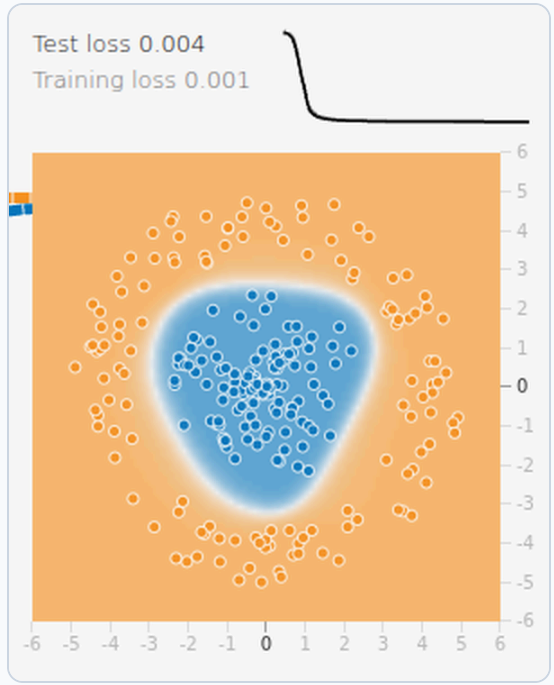
# More layers and neurons mean more capacity




**No hidden layer**
Test loss 0.503. Only a straight line is possible.




**1 layer, 4 neurons**
Test loss 0.010. A curved boundary appears.

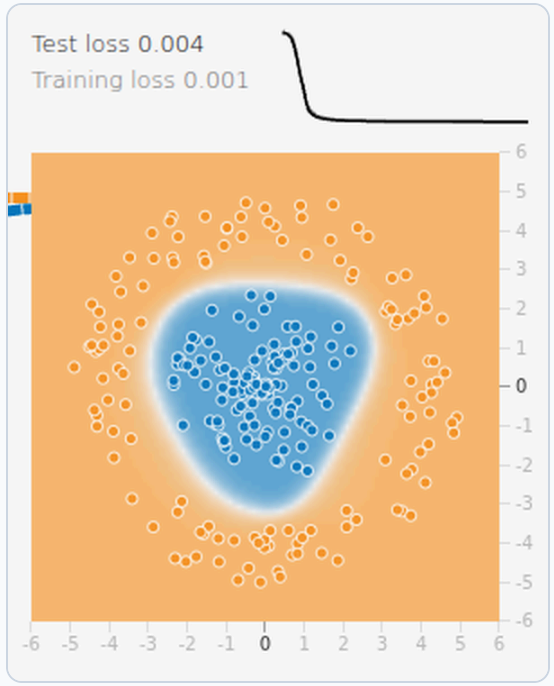



**2 layers, 4 + 2**
Test loss 0.004. Lowest of the three.

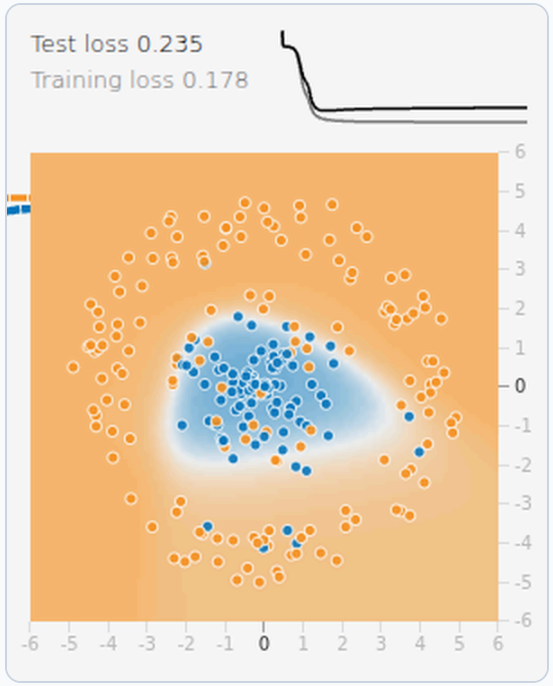
Circle data, tanh, learning rate 0.03, about 500 epochs. A network with one hidden neuron stalled at 0.405. Bigger networks also took about three times longer per epoch in my 8-neuron runs.

PLAYGROUND EXPERIMENT 2

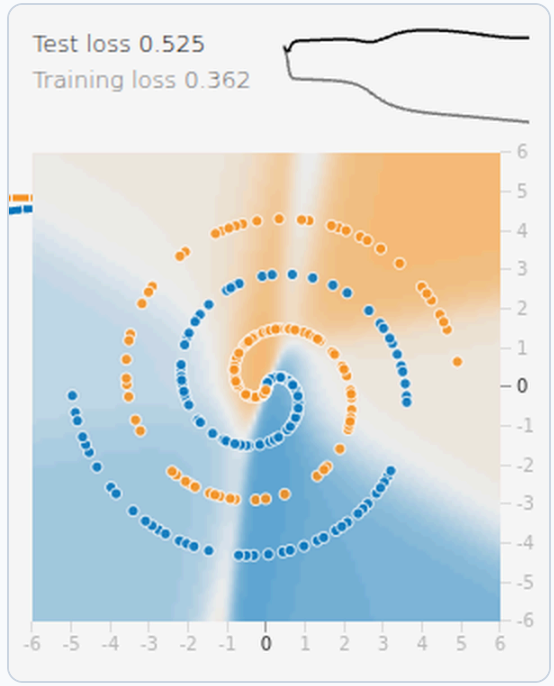
# Noise and harder data




**Circle, noise 0**
Train 0.001, test 0.004




**Circle, noise 40**
Train 0.178, test 0.235

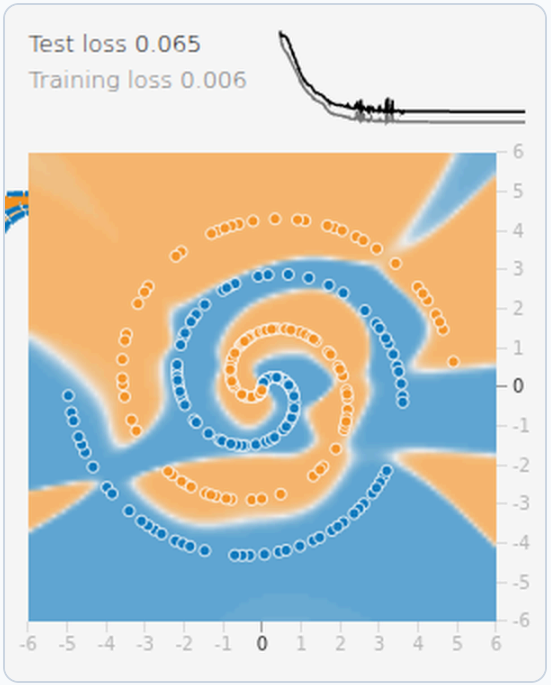



**Spiral, 4 + 2, raw x and y**
Train 0.362, test 0.525

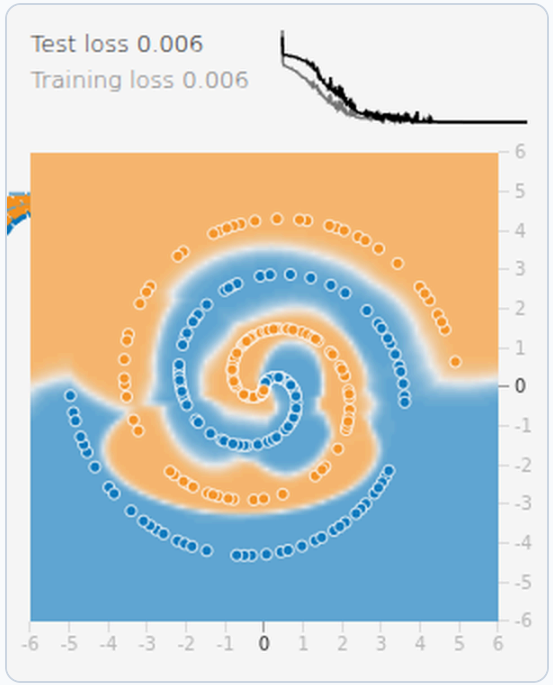
With noise, the classes overlap, so the loss cannot reach zero and the test loss is higher than the training loss. The spiral needs more than a small network and two raw inputs. On XOR data, noise 30 raised test loss from 0.001 to 0.211.

PLAYGROUND EXPERIMENT 3

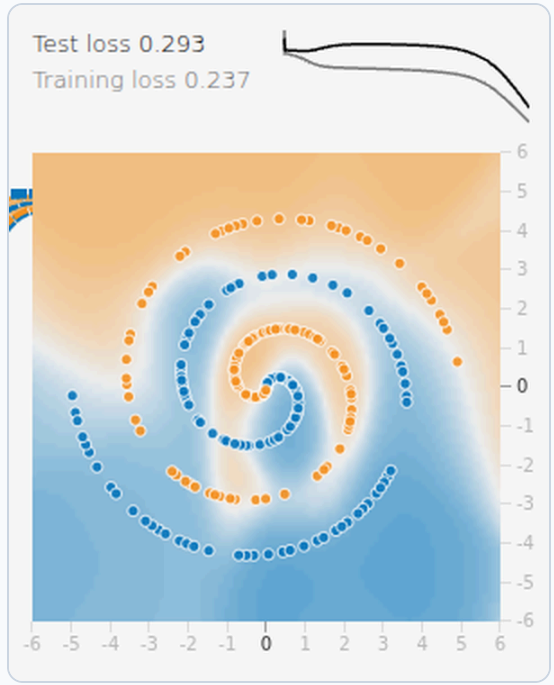
# Activation function and extra features




**Tanh, 8 + 8**
Train 0.006, test 0.065




**ReLU, 8 + 8**
Train 0.006, test 0.006




**Sigmoid, 8 + 8**
Train 0.237, test 0.293

Spiral data with the same two layers of 8 neurons and added features ($x^2$, $y^2$, sin x, sin y), about 503 epochs. Only the activation changed. ReLU fit best and generalised best. Tanh had a gap between train and test, a sign of overfitting. Sigmoid was still learning slowly.

SUMMARY

# Why visualizing neural networks matters

## Why visualize

- A network is just numbers. Pictures make the abstract parts, like weights and layers, visible.
- Seeing the decision boundary form shows *why* a setting works or fails, not only *that* it does.
- It makes ideas easy to explain to people who do not write code.

The Playground is a small 2D toy, so it shows concepts, not how large real systems behave.

## Key insights from my experiments

- **Depth and width** let a network learn curves: 0.503 to 0.004 test loss.
- **Activation** choice mattered a lot: ReLU 0.006 vs sigmoid 0.293.
- **Learning rate** needs balance: 0.001 and 3 both failed, 0.03 worked.
- **Noise** widens the gap between train and test loss (0.004 to 0.235 on the circle).
- **The spiral** needed a bigger network and extra input features together (test loss 0.525 to 0.065).